

Unbeschränkte Minimierung & Berechenbarkeit —Alternative Berechnungsmodelle—

Markus Hecher

Universität Potsdam, Germany

Vorlesung am May 18, 2026

Last updated: 2026-05-17

Thema Heute

1 (Stark wachsende) totale Funktionen außerhalb von $\mathcal{PF}$

~> Cantorsche Paarungsfunktion zur Simulation eines Stacks

- μ -rekursive Funktionen und **unbeschränkte** Suche/Minimierung

~> Äquivalenz zur While-Berechenbarkeit

2 Zurück zur Berechenbarkeit

- Semi-entscheidbarkeit bedeutet bereits Aufzählbarkeit

- Wir zeigen, dass $\mathcal{TCF}_{\mathbb{N}}$ nicht aufgezählt werden kann

~> Damit können totale Funktionen von keinem (zmd. semi-entscheidbaren) Berechnungskonzept erfasst werden

Funktionen außerhalb von $\mathcal{PF}$?

Gibt es totale Funktionen außerhalb von $\mathcal{PF}$?

- Primitiv-rekursiven Funktionen können “nahezu” alle totalen Funktionen berechnen
 - Was fehlt?
- ⇒ Es geht um stark wachsende Funktionen ...

Hierarchie der Standard-Rechenoperationen

- Nachfolgerfunktion succ

- Addition als wiederholte Anwendung der Nachfolgerfunktion

$$n + m = n + \underbrace{1 + \dots + 1}_{m\text{-mal}} = \{\text{für } m \neq 0\} 1 + (n + (m - 1))$$

- Multiplikation als wiederholte Anwendung der Addition

$$n \cdot m = \underbrace{n + \dots + n}_{m\text{-mal}} = \{\text{für } m \neq 0\} n + n \cdot (m - 1)$$

$\rightsquigarrow$ Viel schnelleres Wachstum als Addition

- Potenzierung als wiederholte Anwendung der Multiplikation

$$n^m = \underbrace{n \cdot \dots \cdot n}_{m\text{-mal}} = \{\text{für } m \neq 0\} n \cdot n^{m-1}$$

$\rightsquigarrow$ Viel schnelleres Wachstum als Multiplikation

$\uparrow$ -Operator

- Wie lässt sich diese Hierarchie fortsetzen?

- $$n \uparrow^k m = \begin{cases} n^m & \text{falls } k = 1, \\ 1 & \text{falls } m = 0, \\ n \uparrow^{k-1} (n \uparrow^k (m-1)) & \text{sonst} \end{cases}$$

↪ Je größer k , desto größer das Wachstum

- Idee: Jede primitiv-berechenbare Funktion wird durch ein Level k beschränkt

↪ Mittels Schlüssellemma gilt für alle $f \in \mathcal{PF}$ und $n \geq n_0$, dass $f(n) \leq 2 \uparrow^k (n)$

↪ Funktionen, die quer durch alle Level gehen, sind nicht primitiv-rekursiv

Beispiel: Ackermannfunktion

- i .te Ackermannfunktion $B_i : \mathbb{N} \rightarrow \mathbb{N}$ mit
 - $B_0(n) = \begin{cases} 1 & \text{falls } n = 0, \\ 2 & \text{falls } n = 1, \\ 2 + n & \text{falls } n \geq 2 \end{cases}$
 - $B_{i+1}(0) = 1$
 - $B_{i+1}(n+1) = B_i(B_{i+1}(n))$
- Ackermannfunktion $A : \mathbb{N} \rightarrow \mathbb{N}$ mit $A(n) = B_n(n)$

Ackermannfunktion: Zusammenhang zum $\uparrow$ -Operator

- $B_0(n) = 2 + n$ für $n \geq 2$ (per Definition)

- $B_1(n) = 2 \cdot n$ für $n \geq 1$:

$$B_1(1) = B_0(B_1(0)) = B_0(1) = 2 = 2 \cdot 1$$

$$B_1(n+1) = B_0(B_1(n)) = B_0(2 \cdot n) = 2 + 2 \cdot n = 2 \cdot (n+1)$$

$\rightsquigarrow$ $B_i(n) = 2 \uparrow^{i-1} n$ für $i \geq 2$:

$$\frac{}{B_i(0) = 1 = 2 \uparrow^{i-1} 0}$$

$$B_i(n+1) = B_{i-1}(B_i(n)) = B_{i-1}(2 \uparrow^{i-1} n) = 2 \uparrow^{i-2} (2 \uparrow^{i-1} n) = 2 \uparrow^{i-1} (n+1)$$

Resultat: $A \notin \mathcal{PF}$

Beweis (Grundidee).

■ Annahme $A \in \mathcal{PF}$

1 Schlüssellemma: Für $A \in \mathcal{PF}$ gibt es $i, m \in \mathbb{N}$ mit $\forall n \in \mathbb{N}, A(n) \leq B_i^m(n)$

2 Lemma: Es gibt ein $n_0 \in \mathbb{N}$, sodass $\forall n \geq n_0, B_i^m(n) \leq B_{i+1}(n)$

3 Setze $n = \max(n_0, i + 2)$

4 Lemma (starke Monotonität): $B_n(n) > B_{i+1}(n)$

■ Insgesamt: $A(n) \leq B_i^m(n) \leq B_{i+1}(n) < B_n(n) = A(n)$, ein Widerspruch

⇒ Annahme falsch, also $A \notin \mathcal{PF}$



■ Ackermannfunktion im Wesentlichen $\uparrow$ -Operator mit fester erster Eingabe 2

⇒ übersteigt alle Level, aber dennoch in $\mathcal{TW}\mathcal{F} = \mathcal{TG}\mathcal{F} = \mathcal{TT}\mathcal{F}$!

Funktionen außerhalb von $\mathcal{PF}$?
Aber: ... mit Stack berechenbar!

Schrittweise Berechnung der Ackermannfunktion

- Ackermannfunktion $A \notin \mathcal{PF}$, aber dennoch über Stacks berechenbar:
- Start mit Stack $\langle 2, \langle n, n \rangle \rangle$ (für $A(n) = B_n(n)$)
- Schrittweise Änderung des Stacks gemäß der Berechnungsvorschriften von B_i :
 - $B_n(0) = 1$: $\delta \langle \ell + 2, \langle n_1, \dots, n_\ell, n, 0 \rangle \rangle = \langle \ell + 1, \langle n_1, \dots, n_\ell, 1 \rangle \rangle$
 - $B_0(1) = 2$: $\delta \langle \ell + 2, \langle n_1, \dots, n_\ell, 0, 1 \rangle \rangle = \langle \ell + 1, \langle n_1, \dots, n_\ell, 2 \rangle \rangle$
 - $B_0(n + 2) = n + 4$: $\delta \langle \ell + 2, \langle n_1, \dots, n_\ell, 0, n + 2 \rangle \rangle = \langle \ell + 1, \langle n_1, \dots, n_\ell, n + 4 \rangle \rangle$
 - $B_{n+1}(m + 1) = B_n(B_{n+1}(m))$:
 $\delta \langle \ell + 2, \langle n_1, \dots, n_\ell, n + 1, m + 1 \rangle \rangle = \langle \ell + 3, \langle n_1, \dots, n_\ell, n, n + 1, m \rangle \rangle$

$\rightsquigarrow \delta : \mathbb{N} \rightarrow \mathbb{N}$ ist in $\mathcal{PF}$

Die unbeschränkte Suche

- Anzahl der Zahlen im Stack oder erste bearbeitete Zahl wird kleiner
 - Gibt kleinstes $k \in \mathbb{N}$, so dass $\pi_1^2(\delta^k \langle 2, \langle n, n \rangle \rangle) = 1$
- $\rightsquigarrow A(n) = \pi_2^2(\delta^k \langle 2, \langle n, n \rangle \rangle)$
- Problem: Berechnung von k nicht primitiv-rekursiv
- $\rightsquigarrow$ Brauchen Suche wie $Mn[f]$ aber ohne Obergrenze für Suchschritte

μ -Rekursive Funktionen

Definition

- Alle primitiv-rekursiven Grundfunktionen auch μ -rekursiv
- Abschluss unter Komposition und primitiver Rekursion (wie zuvor)

- Zusätzlich unbeschränkte Minimierung $\mu f : \subseteq \mathbb{N}^{\ell} \rightarrow \mathbb{N} \in \mu\mathcal{F}$ für
 $f : \subseteq \mathbb{N}^{\ell+1} \rightarrow \mathbb{N} \in \mu\mathcal{F}$ mit

$$\mu f(n_1, \dots, n_{\ell}) = \begin{cases} \min\{m \mid f(n_1, \dots, n_{\ell}, m) = 0\} & \begin{array}{l} \text{falls dies existiert und} \\ f(n_1, \dots, n_{\ell}, m') \neq \perp \\ \text{für alle } m' < m, \end{array} \\ \perp & \text{sonst} \end{cases}$$

- Kurzform ' $\text{fe} < \text{def}(n_1, \dots, n_{\ell}, m)$ ' für 'falls das Minimum existiert und $f(n_1, \dots, n_{\ell}, m') \neq \perp$ für alle $m' < m$ '

μ -Berechenbarkeit

- Wie bei primitiv-rekursiven Funktionen kein Unterschied zwischen Syntax und Semantik, d. h. μ -berechenbare Funktionen genau die μ -rekursiven Funktionen
- ↪ $\mu\mathcal{F}$ = die Menge aller μ -berechenbaren Funktionen
- ↪ $\mathcal{T}\mu\mathcal{F}$ = die Menge aller totalen μ -berechenbaren Funktionen

Beispiele

$$\blacksquare \mu \text{const}_1^2(n) = \begin{cases} \min\{m \mid \text{const}_1^2(n, m) = 0\} & \text{falls } n = 0, \\ \perp & \text{sonst} \end{cases} = \perp$$

$$\blacksquare \mu \text{add}(n) = \begin{cases} 0 & \text{falls } n = 0, \\ \perp & \text{sonst} \end{cases}$$

$$\blacksquare \text{Sei } h(n, m) = 0 \text{ falls } n = m \text{ und } h(n, m) = \perp \text{ sonst}$$

$$\mu h(n) = \begin{cases} 0 & \text{falls } n = 0, \\ \perp & \text{sonst} \end{cases}$$

$\rightsquigarrow$ Haben stets $h(n, n) = 0$, aber $h(n, m) = \perp$ für $m < n$

Äquivalenz zur While-Berechenbarkeit

Äquivalenz zur While-Berechenbarkeit

$$\mathcal{WF} \subseteq \mu\mathcal{F}$$

$$\mathcal{WF} \subseteq \mu\mathcal{F} \mid$$

- Übernahme der Beweisstruktur und aller (while-losen) Konstrukte aus Beweis für $\mathcal{LF} \subseteq \mathcal{PF}$
- ↪ Nur Mittelteil ändern, bei dem der aktuelle Speichervektor (codiert in einer einzelnen Zahl) verändert wird
- P_f von der Form **while** x_i **do** P' **end** mit zugehöriger Funktion $g_{P'} : \subseteq \mathbb{N} \rightarrow \mathbb{N} \in \mu\mathcal{F}$, die den Speichervektor $\langle m_0, \dots, m_{\max\{\ell, l_{P_f}\}} \rangle$ entsprechend ändert
- ↪ Finde zunächst Anzahl nötiger Wiederholungen
- ↪ Führe dann $g_{P'}$ entsprechend oft aus

$$\mathcal{WF} \subseteq \mu\mathcal{F} \parallel$$

- $h : \subseteq \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $h(n, m) = \pi_{i+1}^{\max\{\ell, l_{P_f}\}+1}(g_{P'}^m(n)) \in \mu\mathcal{F}$
- $\mu h : \subseteq \mathbb{N} \rightarrow \mathbb{N}$ mit $\mu h(n) = \begin{cases} \min\{m \mid h(n, m) = 0\} & \text{fe} < \text{def}(n, m), \\ \perp & \text{sonst} \end{cases}$

↪ Gesuchte Veränderung des Speichervektors gemäß der While-Schleife durch

$$g_1 : \subseteq \mathbb{N} \rightarrow \mathbb{N} \text{ mit } g_1 \langle m_0, \dots, m_{\max\{\ell, l_{P_f}\}} \rangle = \\ g_{P'}^{\mu h \langle m_0, \dots, m_{\max\{\ell, l_{P_f}\}} \rangle} \langle m_0, \dots, m_{\max\{\ell, l_{P_f}\}} \rangle$$

Kleenesche Normalform für μ -rekursive Funktionen

Korollar

Für jede ℓ -stellige $f \in \mu\mathcal{F}$ gibt es $g, h \in \mathcal{PF}$, sodass $f(n_1, \dots, n_\ell) = g(n_1, \dots, n_\ell, \mu h(n_1, \dots, n_\ell))$.

- Starte von While-Programm in Kleenescher Normalform
 - Führe die Konstruktionen vom Beweis von $\mathcal{WF} \subseteq \mu\mathcal{F}$ und $\mathcal{LF} \subseteq \mathcal{PF}$ durch
- ↪ Dann wird nur eine unbeschränkte Suche benutzt

Äquivalenz zur While-Berechenbarkeit

$$\mu\mathcal{F} \subseteq \mathcal{WF}$$

$\mu\mathcal{F} \subseteq \mathcal{WF}$

- Wiederverwendung von Beweis $\mathcal{PF} \subseteq \mathcal{LF}$ für alles außer unbeschränkte Suche
- $f :_{\subseteq} \mathbb{N}^{\ell+1} \rightarrow \mathbb{N}$ mit $f \in \mu\mathcal{F}$ vorausgesetzt

↪ Benötigen While-Programm für $\mu f :_{\subseteq} \mathbb{N}^{\ell} \rightarrow \mathbb{N}$ mit

$$\mu f(n_1, \dots, n_{\ell}) = \begin{cases} \min\{m \mid f(n_1, \dots, n_{\ell}, m) = 0\} & \text{falls } f \text{ definiert } (n_1, \dots, n_{\ell}, m), \\ \perp & \text{sonst} \end{cases}$$

- Aufruf von While-berechenbaren Funktionen in While-Programmen analog zu den Aufrufen in Loop-Programmen

$\mu\mathcal{F} \subseteq \mathcal{WF} \parallel$

- Für $\mu f :_{\subseteq} \mathbb{N}^\ell \rightarrow \mathbb{N}$ mit

$$\mu f(n_1, \dots, n_\ell) = \begin{cases} \min\{m \mid f(n_1, \dots, n_\ell, m) = 0\} & \text{fe} < \text{def}(n_1, \dots, n_\ell, m), \\ \perp & \text{sonst} \end{cases}$$

- ... bauen wir:

$y := f(x_1, \dots, x_\ell, 0);$

while $y \neq 0$ **do**

$x_0 := \text{suc}(x_0);$

$y := f(x_1, \dots, x_\ell, x_0);$

end

Äquivalenz zur While-Berechenbarkeit

Zusammenfassung bisheriger Ergebnisse

Zusammenfassung bisheriger Ergebnisse

- $\mathcal{PF} = \mathcal{LF} \subset (\mathcal{T}\mu\mathcal{F} = \mathcal{T}\mathcal{GF} = \mathcal{T}\mathcal{WF} = \mathcal{T}\mathcal{TF}) \subset (\mu\mathcal{F} = \mathcal{GF} = \mathcal{WF} = \mathcal{TF})$
- Viele Modelle, die zur Charakterisierung der Berechenbarkeit in Frage kommen
- ⇒ (Bisher) kein Modell zur Charakterisierung der totalen Berechenbarkeit

Zurück zur Berechenbarkeit

Zurück zur Berechenbarkeit

Beobachtungen

Beschränkung auf Funktionen vom Typ $\mathbb{N} \rightarrow \mathbb{N}$

- (Semi-)Entscheidbarkeit durch (partiell) charakteristische Funktion ersetzbar
- Berechenbarkeit auf Wörtern durch Berechenbarkeit auf natürliche Zahlen simulierbar
 - Wörter durch Zahlen codierbar
 - Wörter durch Zahlen systematisch nummerierbar
 - Arbeit mit Zahlen einfacher
 - Auch Programme als Zahlen codierbar (folgt)
- Einstellige Funktionen wegen Cantorscher Paarungsfunktion ausreichend
 - $f :_{\subseteq} \mathbb{N}^{\ell} \rightarrow \mathbb{N}^k$ simulierbar durch $f' :_{\subseteq} \mathbb{N} \rightarrow \mathbb{N}$ mit
$$f' \langle n_1, \dots, n_{\ell} \rangle = \langle f_1(n_1, \dots, n_{\ell}), \dots, f_k(n_1, \dots, n_{\ell}) \rangle$$

Turingmaschinen als Wörter codierbar

- Ausreichend, Turingmaschinen M der Form $(\{q_0, \dots, q_n\}, \{1\}, \{1, \#, B\}, \delta, q_0, \{q_1\})$ zu betrachten
- Definiere $\text{code}(\delta(q, X))$ als das Wort $qXpYD$, falls $\delta(q, X) = (p, Y, D)$, und als das Wort ε , falls $\delta(q, X) = \perp$

↪ Codiere M durch das Wort

$\text{code}(\delta(q_0, 1)) \text{code}(\delta(q_0, \#)) \text{code}(\delta(q_0, B)) \dots \text{code}(\delta(q_n, B))$

- Codiere Alphabet $\{q_0, \dots, q_{n-1}, 1, \#, B, L, R\}$ wie folgt als Wörter über $\Delta = \{0, 1\}$:
 $1 \mapsto 10, \# \mapsto 100, B \mapsto 1000, L \mapsto 10^4, R \mapsto 10^5, q_i \mapsto 10^{i+6}$
- Wende Alphabetscodierung auf den Code von M an

WARUM?

↪ Bijektive Nummerierung von Turingmaschinen

- Turingmaschinen als Wörter über $\{0, 1\}$ codierbar (s. o.)
- Lexikographische Ordnung $\varepsilon < 0 < 1 < 00 < 01 < 10 < 11 < 000 < \dots$
(Länge, dann binärer Wert als Zweitkriterium)
- Einige dieser Binärfolgen repräsentieren TMs, andere nicht
 - ↪ Bilde Teilfolge, sodass nur Binärfolgen stehen bleiben, die eine TM repräsentieren
- ↪ M_i ist diejenige Maschine, deren Codierung an i .ter Position in dieser Teilfolge
 - i heißt **Gödelnummer** von M_i
 - Die hier beschriebene **Gödelisierung** ist eine bijektive Funktion vom Typ $\text{TMs} \rightarrow \mathbb{N}$, wobei die zuvor genannten "Restriktionen" an die TMs gelten

Aufzählbarkeit berechenbarer Funktionen

- Sei $\varphi_i : \subseteq \mathbb{N} \rightarrow \mathbb{N}$ definiert durch
$$\varphi_i(n) := \begin{cases} r_u^{-1}(f_{M_i}(r_u(n))) & \text{falls } f_{M_i}(r_u(n)) \neq \perp, \\ \perp & \text{sonst} \end{cases}$$
- Dann existiert für jedes $f : \subseteq \mathbb{N} \rightarrow \mathbb{N} \in \mathcal{CF}$ ein $i \in \mathbb{N}$, so dass $f = \varphi_i$
 - d. h. totale Funktion $\varphi : \mathbb{N} \rightarrow \mathcal{CF} \cap (\subseteq \mathbb{N} \rightarrow \mathbb{N})$ mit $\varphi(i) = \varphi_i$ ist surjektiv
 - φ aber nicht injektiv, da verschiedene TMs dieselbe Funktion berechnen können
- Definiere ferner die Rechenzeitfunktion $\Phi_i : \subseteq \mathbb{N} \rightarrow \mathbb{N}$ durch
$$\Phi_i(n) := \begin{cases} t_{M_i}(r_u(n)) & \text{falls } f_{M_i}(r_u(n)) \neq \perp, \\ \perp & \text{sonst} \end{cases}$$

↪ t_{M_i} gibt an, wie viele Schritte M_i auf Eingabe $r_u(n)$ braucht

Wichtige Beobachtungen

1 Es gibt ein $j \in \mathbb{N}$, sodass $\varphi_j \langle i, n \rangle = \varphi_i(n)$

Dieses φ_j wird **universelle Funktion** **u** genannt, also **$u \langle i, n \rangle = \varphi_i(n)$**

2 Es gibt ein $j \in \mathbb{N}$, sodass $\chi_{\{\langle i, n, t \rangle \mid \Phi_i(n) = t\}} = \varphi_j$,

d. h. **$\{\langle i, n, t \rangle \mid \Phi_i(n) = t\}$ entscheidbar**

1. Universelle Funktion $u\langle i, n \rangle = \varphi_i(n)$ berechenbar

Konstruktionsidee.

↪ Konstruiere eine zu u gehörige universelle TM...

- Rekonstruktion von i und n (in unärer Darstellung) aus der Eingabe
- Rekonstruktion der Überföhrungsfunktion von M_i aus i (durch Nacheinanderaufzählung aller Bitfolgen, die TMs darstellen)
- Simulationsphase mit mindestens drei Bändern:
 - Für δ_{M_i}
 - Für das aktuelle Band von M_i (startet mit n)
 - Für den aktuellen Zustand von M_i
- Genau dann, wenn M_i eine gültige Ausgabe berechnet, soll auch die konstruierte Maschine eine gültige Ausgabe berechnen



2. Menge $\{\langle i, n, t \rangle \mid \Phi_i(n) = t\}$ und Rechenzeitfunktion entscheidbar

Konstruktionsidee.

- Konstruiere M_j wie folgt...
 - Teile Eingabe $\langle i, n, t \rangle$ in $\langle i, n \rangle$ und t
 - Führe die zu u gehörende Maschine auf $\langle i, n \rangle$ für maximal t Schritte aus (Realisierung durch Dekrementieren von t)
 - Endet die Simulation vor t Schritten, gib 0 aus
 - Endet die Simulation nach genau t Schritten, gib 1 aus
 - Hat die Simulation noch nicht geendet, gib 0 aus

